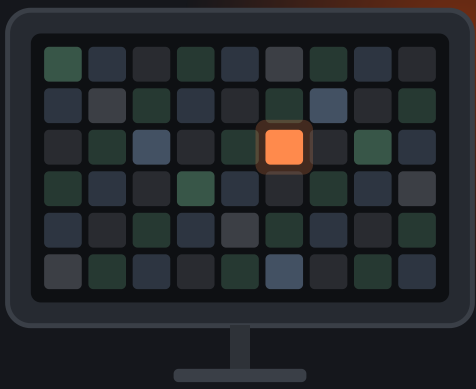
MANUAL DE CAMPO · LOTES 1 A 3 · PRÉVIA



Estado & Tela

Aplicações web reativas com React, do DOM manipulado à mão até uma aplicação completa, autenticada e testada, construída com agentes de IA sem perder o controle do código.

ORGANIZADO POR

Prof. Rogério Silva e Claude Opus 5.5

2026 · TEC.1052 · ADS · IFPI

React 19.3 · Vite 8 · TypeScript
React Router 8 · TanStack Query 5 · Vitest 5 · Playwright

Sumário

0 Introdução P. 3

PARTE I · A MUDANÇA DE PARADIGMA

1 Do DOM à mão às interfaces reativas P. 7

2 O primeiro projeto em vinte minutos P. 18

3 Como o React funciona por dentro P. 28

PARTE II · OS FUNDAMENTOS

4 Componentes, JSX e composição P. 41

5 Props, eventos e fluxo de dados P. 52

6 Estado, formulários e interações P. 62

7 Efeitos: sincronizar com o mundo lá fora P. 75

PARTE III · ARQUITETURA

8 Estrutura de projeto por funcionalidade LOTE 4

9 Estilo, tokens e acessibilidade LOTE 4

10 Componentes reutilizáveis e design system LOTE 4

11 Prop drilling e Context LOTE 5

12 Reducer e transições de estado LOTE 5

PARTE IV · A APLICAÇÃO COMPLETA

13 Rotas e navegação LOTE 6

14 Backend, HTTP e estado remoto LOTE 6

15 Autenticação, sessão e autorização na interface LOTE 7

16 Testes, depuração e desempenho LOTE 7

PARTE V · RENDERIZAÇÃO MODERNA

17 Servidor, cliente e a fronteira entre eles LOTE 8

18 Next.js: visão geral e próximo livro LOTE 8

PARTE VI · ENTREGA E REFERÊNCIA

19 A entrega com agentes, do plano ao PR LOTE 9

20 Guia de consulta, régua e glossário LOTE 9

ANTES DE COMEÇAR

00 Introdução

Para quem é este livro, a ideia que ele defende e como cada capítulo funciona.

Você já sabe fazer uma página reagir a um clique. Este livro trata do passo seguinte: fazer uma aplicação inteira continuar certa quando ela tiver trinta telas, várias pessoas no código e um agente de IA escrevendo boa parte dele.

É para quem já constrói páginas com HTML, CSS e JavaScript, manipula o DOM, consome uma API com `fetch` e já escreveu um backend REST com Express e TypeScript. Nada disso é ensinado de novo. Se você cursa TEC.1052, guarde por perto o front-end do Mini-Prontuário que escreveu em JavaScript puro: ele é o ponto de partida do capítulo 1.

> **A tela é uma função do estado. Quase todo bug de front-end é um estado guardado no lugar errado, duplicado ou sincronizado à mão.**

Por isso o livro não é uma lista de APIs do React. Cada capítulo responde, de um ângulo diferente, a uma pergunta: **onde este estado mora?**



Fig. 0.1 — Os quatro lugares onde um estado pode morar. Hooks, contexto, reducer, rotas e bibliotecas aparecem no livro como respostas a esta pergunta.

O laboratório

> H+ **O livro inteiro acontece num hospital**
>
> Ao lado de cada leito há um monitor. Ninguém redesenha a tela dele à mão: os sensores mudam um número e o monitor atualiza só a faixa que mudou. É o modelo mental do React, e o mundo de todas as analogias do livro.

O projeto é o cliente React do **Mini-Prontuário**, que conversa com a API do ebook *Crachá & Porta*: sessão em cookie, permissões por papel e uma suíte de ataques. A interface é nova; o servidor é real desde o primeiro dia. São três repositórios, `estado-e-tela-api`, `estado-e-tela-web` (uma tag git por capítulo) e `estado-e-tela-ebook`, com este livro, os roteiros de medição e as saídas. Todos os pacientes são fictícios.

Como cada capítulo funciona

A ordem é sempre a mesma: o **problema** num sintoma do Mini-Prontuário; o **modelo mental**, com a caixa *Onde a analogia para*; o mecanismo **por dentro** e **na prática**; o **erro clássico** e a **prova**, um teste que falha sem a correção; o trabalho **com agentes de IA**; e o **fechamento**, com exercícios em três níveis.

Este livro faz parte de uma série. O *Desenvolvimento Dirigido por IA*, o Dev de IA, ensina o método de trabalhar com agentes; aqui ele é aplicado ao React,

sem repetição. Quando um procedimento está detalhado lá, uma caixa **Ponte · Dev de IA** resume e aponta o capítulo. Os prompts seguem os formatos da disciplina:

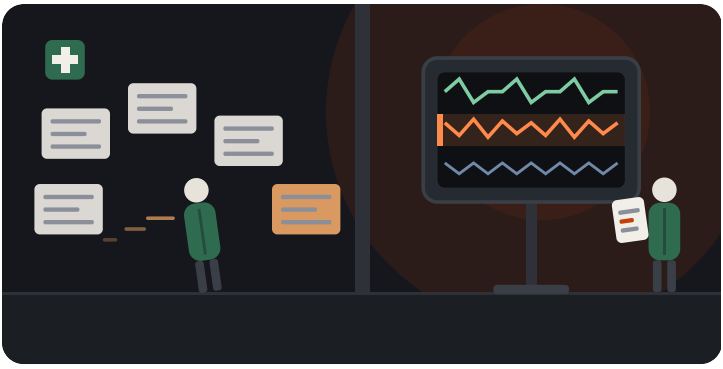
FORMATO	QUANDO	PARTES
Agente	a IA edita o projeto	Goal · Context · Constraints · Done when
C-P-T-R-F-A	conversa no chat	Contexto · Papel · Tarefa · Restrições · Formato · Aceitação
V.E.R.M.	antes de aceitar a resposta	Verifica · Explica · Reduz · Marca

O livro traz o prompt de agente; as variantes de cada capítulo ficam em `prompts/capNN.md`, no repositório da tela.

A régua e as convenções

O capítulo 20 lista **36 competências** de um desenvolvedor sênior de front-end, cada uma com a evidência exigida. Marcar 27 com evidência é chegar a **75%**. Cada capítulo diz quais itens entrega.

- **Dado** é fato externo, com fonte. **Medido** é saída real de código que rodamos, guardada em `saidas/`; nenhuma foi escrita à mão.
- Todo código impresso sai do git, na tag indicada na barra do bloco, em linhas de até 46 caracteres.



PARTE I · A MUDANÇA DE PARADIGMA

01

Do DOM à mão às interfaces reativas

O trabalho de manter a tela em dia com os dados não some quando você adota React. Ele muda de dono.

Terça, 14h. A enfermeira dá alta para Maria Souza, paciente de alto risco da Ala B. A linha some, o total cai de 6 para 5, e o painel continua dizendo: *alto risco: 2*.

Ninguém escreveu um erro; escreveram uma atualização a menos. A tela tinha três lugares que dependiam da mesma lista, e o código da alta lembrou de dois.

1.1

Como chegamos aqui

Em 2005, o AJAX deixou a página pedir dados sem recarregar. Em 2006, o jQuery tornou a manipulação do DOM o jeito padrão de programar a tela. Em 2010, Backbone e AngularJS popularizaram as aplicações de página única. Em 2013 veio o React, com uma ideia diferente: a pergunta deixou de ser *como alterar a tela* quando um dado muda e passou a ser *como a tela deve ficar* para um estado qualquer. Vue e Svelte seguiram a mesma ideia com outra mecânica, e em 2026 o React passou a ser mantido pela React Foundation, na Linux Foundation.

1.2

A Ala B em três versões

O laboratório está em `laboratorio/cap01`, na tag `cap01`: três páginas com a mesma tela, sem build. Seis pacientes fictícios, uma busca, um campo de anotação e um botão de alta por paciente, e um resumo com o total e quantos são de alto risco. Sirva a pasta com `npx serve laboratorio/cap01` e abra cada página.

Cada uma traz no topo um **medidor**, um `MutationObserver` que conta os elementos que entram e saem do DOM e os textos ou atributos trocados. É ele que transforma "o React é eficiente" em número.

1.3

Versão 1: você diz como mudar

A primeira versão é **imperativa**: o código dá ordens ao DOM, passo a passo. Na montagem ele preenche o total e o alto risco; na alta, remove a linha e corrige o total:

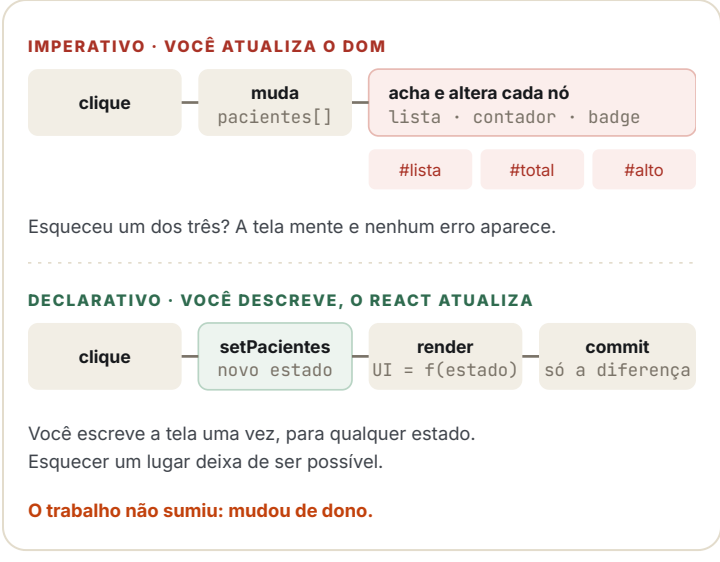


Fig. 1.1 — Quem atualiza a tela. No imperativo, cada evento precisa encontrar e alterar cada nó afetado. No declarativo, você descreve a tela para qualquer estado, e o React calcula o que mudar.

1-imperativo.html · alta cap01

```
// alta: tira a linha e corrige o total
lista.addEventListener("click", (e) => {
  if (e.target.tagName !== "BUTTON") return;
  const li = e.target.closest("li");
  pacientes = pacientes
    .filter((p) => p.id !== li.dataset.id);
  li.remove();
  total.textContent = pacientes.length;
});
```

O código precisa saber quais nós existem, quais dependem de `pacientes` e em que ordem atualizá-los. Com três dependências já houve esquecimento; uma tela real tem dezenas.

1.4

Versão 2: estado e `render()`

A segunda versão é a que a disciplina ensinou: um objeto `estado` e uma função que redesenha a tela inteira a partir dele.

2-render.html · render() cap01

```
// a tela inteira sai do estado, sempre
function render() {
  const t = estado.busca.toLowerCase();
  const vis = estado.pacientes.filter((p) =>
    p.nome.toLowerCase().includes(t));
  const alto = estado.pacientes
    .filter((p) => p.risco === "alto").length;
  app.innerHTML = `
    <p>${estado.pacientes.length} pacientes
```

```
      · alto risco: ${alto}</p>
    <ul>${vis.map((p) => `
      <li data-id="${p.id}">
        <b>${p.nome}</b> ${p.leito}
        <input placeholder="anotação">
        <button>Alta</button>
      </li>`).join("")}
    </ul>`;
}
```

Ela já pensa do jeito certo: o alto risco é **calculado** a cada render e nunca fica errado. O problema é o `innerHTML`, que joga fora todos os nós e cria outros. Cada tecla na busca reconstrói a lista, e o que estava dentro dos nós antigos vai junto: o texto das anotações, o foco.

1.5

Versão 3: React

A terceira mantém a ideia da segunda e troca o mecanismo. O React vem do `esm.sh` por um `importmap`:

3-react.html · Ala — cap01

```
// a tela é uma função do estado
function Ala() {
  const [busca, setBusca] = useState("");
  const [pacientes, setPacientes] =
    useState(PACIENTES);

  const t = busca.toLowerCase();
  const vis = pacientes.filter((p) =>
    p.nome.toLowerCase().includes(t));
```

```
  const alto = pacientes
    .filter((p) => p.risco === "alto").length;
  const alta = (id) => setPacientes(
    pacientes.filter((p) => p.id !== id));

  return h("div", null,
    h("input", {
      placeholder: "buscar", value: busca,
      onChange: (e) =>
        setBusca(e.target.value),
    }),
    h("p", null,
      `${pacientes.length} pacientes`
      + ` · alto risco: ${alto}`),
    h("ul", null, vis.map((p) =>
      h("li", { key: p.id },
        h("b", null, p.nome), ` ${p.leito} `,
        h("input",
          { placeholder: "anotação" }),
        h("button",
          { onClick: () => alta(p.id) },
          "Alta")))));
}
```

- `useState` guarda um valor entre renders e devolve a função que o troca. `setPacientes` não mexe no DOM: avisa o React de que o estado mudou.
- `h` é o `createElement` do React. Ele devolve uma *descrição* do que deveria existir, não um nó. O JSX vira exatamente estas chamadas (capítulo 3).
- `key: p.id` diz qual paciente cada linha representa.

A função `Ala` não sabe se roda pela primeira ou pela centésima vez: recebe o estado e devolve a tela. Quem

compara a descrição nova com a anterior e aplica a diferença é o React.

1.6

O que o medidor mostra

As três páginas passaram pelo mesmo roteiro num Chromium: anotar "ver febre" em João Lima, digitar `o` na busca, limpar a busca e dar alta a Maria Souza.

```
medicoes/cap01.py · Medido

Chromium 141.0.7390.37

versão     busca o     alta Maria  nota    alto
imperativo +0 -0 ~2    +0 -4 ~1    fica    2
render()   +18 -26 ~0  +22 -26 ~0  some    1
React      +0 -8 ~1    +0 -4 ~1    fica    1

+ elementos que entram  - que saem
~ textos ou atributos trocados
```

- **Busca.** O imperativo esconde duas linhas (dois atributos `hidden`). O `render()` destrói 26 elementos e cria 18. O React remove só as duas linhas que saíram e troca um atributo, o `value` da busca.
- **Alta.** Imperativo e React fazem o mesmo trabalho no DOM, e o imperativo deixa o alto risco em 2.
- **Nota.** Só o `render()` apagou a anotação.

> **O imperativo economiza e erra. O render() acerta e destrói. O React acerta e economiza.**

Eficiente não quer dizer mais rápido: na alta, o React ainda executou `Ala` e comparou duas descrições da tela. O ganho é deixar de escrever, e de esquecer, cada atualização à mão. O capítulo 16 mede quando o tempo passa a importar.

1.7

Declarativo, estado e reatividade

Declarativo é descrever o resultado para um estado e deixar o mecanismo decidir os passos. **Estado da interface** são os dados que, se mudarem, mudam o que aparece. **Reatividade** é a tela acompanhar o estado sem você escrever a atualização. No React ela é explícita: a tela só muda quando você chama o `set` do estado e o componente roda de novo. Vue, Svelte e Solid rastreiam sozinhos o que cada trecho da tela lê; são escolhas diferentes para o mesmo objetivo.

H+ **O monitor de beira-leito**

A enfermeira não redesenha a tela do monitor. Os sinais vitais mudam e o monitor atualiza a faixa que mudou. Os sinais vitais são o estado; a tela do monitor é a interface.

ONDE A ANALOGIA PARA

O monitor real lê os sensores o tempo todo. O React não observa nada: só calcula a tela de novo quando alguém chama o `set` de um estado. Mudar uma variável comum não muda a tela.

1.8

Biblioteca, não framework

O React transforma estado em interface. Rotas, busca de dados, build de produção e renderização no servidor vêm de outras ferramentas ou de um **framework** construído sobre ele.

DADO · REACT.DEV, FEV. 2025

O Create React App foi descontinuado. A documentação recomenda frameworks (Next.js, React Router, Expo) para produção e ferramentas como o Vite para montar a aplicação do zero e aprender o básico.

Este livro escolhe o caminho do zero com Vite de propósito: cada peça entra quando o problema dela aparece, e você vê o que um framework faz antes de delegar. O Next.js aparece nos capítulos 17 e 18 e terá um ebook próprio.

1.9

Com agentes de IA

ARMADILHA DE IA

Dois vícios imperativos aparecem em código gerado: `document.querySelector` dentro de componente para "atualizar um contador", que briga com o React pelo mesmo DOM; e HTML montado em string com `dangerouslySetInnerHTML`, que reabre a porta para XSS que o React fecha ao escapar todo texto.

PROMPT · AGENTE · CAP 1

```
Goal
Reescrever a tela de pacientes do meu
front-end vanilla como função do estado,
com React por importmap, sem build.

Context
@laboratorio/cap01/3-react.html (modelo)
@<sua tela vanilla>.html

Constraints
- sem document.querySelector em componente
- sem innerHTML e sem
  dangerouslySetInnerHTML
- contagens calculadas no render
- key de lista é o id do registro
- fora de escopo: estilo, API, build

Done when
- anotar e buscar: a anotação fica
- após a alta, todas as contagens batem
```

```
- no medidor, a alta só remove a linha
  do paciente e troca textos
```

Exercícios

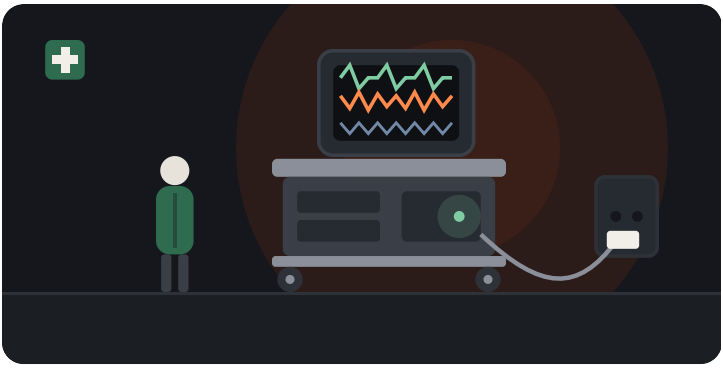
ESSENCIAL Corrija a alta na versão imperativa. Depois liste o que precisaria mudar se a tela ganhasse um terceiro número, como pacientes em isolamento.

INTERMEDIÁRIO Na versão `render()`, mova a busca para dentro do `#app` e explique, com o medidor, por que o campo perde o foco a cada tecla.

DESAFIO Na versão React, troque `key: p.id` por `key: i`. Anote em João Lima, dê alta a Maria Souza e descreva o que aconteceu. O capítulo 3 explica.

Régua: 01. **Tag:** `cap01`. **Prompts:** revisão e C-P-T-R-F-A em `prompts/cap01.md`.

Fontes. react.dev: *Sunsetting Create React App* (fev. 2025), *Creating a React App*, *Introducing the React Foundation* (out. 2025; fundação lançada em fev. 2026). MDN: *MutationObserver*. Medição em `saidas/c1-ala.txt`, Chromium 141, com React 19.3.1 servido localmente nas URLs do importmap.



PARTE I · A MUDANÇA DE PARADIGMA

02

O primeiro projeto em vinte minutos

Do terminal vazio a um projeto versionado, ligado à API e pronto para um agente trabalhar nele.

A página do capítulo 1 funcionou sem build, mas não escala: sem JSX, sem TypeScript, sem pacotes, sem build de produção. Este capítulo monta o projeto de verdade.

Você precisa do **Node 24** (ou 22.22+), Git, um editor e a extensão **React Developer Tools**. O Vite 8 aceita Node 20.19+, mas o React Router 8, que entra no capítulo 13, exige 22.22+; começar no 24 evita trocar de versão no meio do livro.

2.1

Gerando o projeto

terminal

```
npm create vite@latest mini-prontuario-web \
  -- --template react-ts
cd mini-prontuario-web && npm install
npm run dev     # http://localhost:5173
```

O `index.html` é a única página: um `<div id="root">` e o script de `src/main.tsx`, que cria a raiz do React. O `vite.config.ts` configura o Vite, os `tsconfig*.json` os tipos, e o `.oxlintrc.json` o linter, já com as regras dos Hooks.

DADO · O TEMPLATE DE OUTUBRO DE 2026

Com `create-vite` 9.3.1, o template usa **oxlint** em vez de ESLint e fixa o **TypeScript 6.0**: o 7.0, com compilador nativo, ainda não tem a API que parte das ferramentas usa. Não há `"strict": true` porque o modo estrito virou o padrão:

terminal · Medido

```
$ npx tsc -p tsconfig.app.json --noEmit
src/_t.ts(1,19): error TS7006: Parameter 'x'
  implicitly has an 'any' type.
```

2.2

Scripts e o portão

`dev` serve a tela e a atualiza quando você salva; `build` confere os tipos e gera `dist/`; `preview` serve o build; `lint` roda o oxlint. O projeto acrescenta `gate`, que roda tudo o que precisa passar antes de qualquer entrega:

```
terminal · Medido

$ npm run gate
> tsc -b && oxlint && vite build

vite v8.3.5 building client environment for
production...
✓ 16 modules transformed.
…
index-DeCkmCJv.js   219.85 kB | gzip: 68.76 kB
✓ built in 108ms
```

O `tsc -b` não gera arquivos (o `tsconfig` tem `noEmit`): quem transforma TypeScript em JavaScript é o Vite, e ele não confere tipos. Por isso o `dev` mostra a tela mesmo com erro de tipo, e o `gate` não deixa esse erro passar.

2.3

O ponto de entrada

src/main.tsx cap02

```tsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App.tsx";

createRoot(document.getElementById("root")!)
  .render(
    <StrictMode>
      <App />
    </StrictMode>,
  );
```

`createRoot` liga o React a um nó do DOM, e `render` faz o primeiro desenho. `StrictMode` só age em desenvolvimento: chama cada componente duas vezes por render e monta e desmonta cada efeito uma vez a mais, para revelar código que depende de rodar uma vez só. Os capítulos 3 e 7 mostram o que ele pega.

2.4

A tela e a API na mesma origem

A tela roda em `:5173` e a API em `:3000`. Portas diferentes são **origens** diferentes para o navegador, o que traria CORS e regras de cookie entre sites. O **proxy** do Vite evita isso:

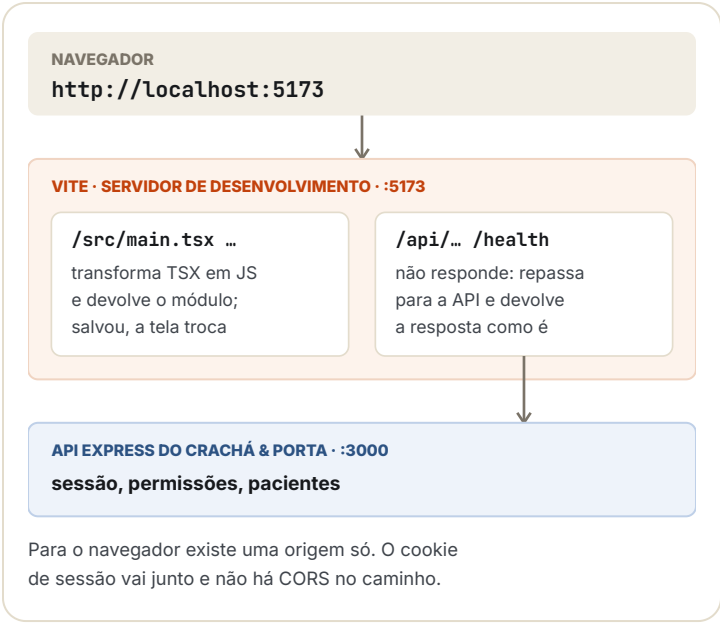


Fig. 2.1 — O navegador só conhece `localhost:5173`. O Vite serve o código da tela e repassa para a API o que começa com `/api` ou `/health`.

vite.config.ts cap02

```ts
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";

// O Vite serve a tela em :5173 e repassa para
// a API em :3000 tudo que começa com /api.
// Para o navegador, é tudo uma origem só.
export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      "/api": "http://localhost:3000",
      "/health": "http://localhost:3000",
    },
```

```
  },
});
```

Em produção não há Vite: o build vira arquivos estáticos e o servidor que os publica faz o papel do proxy. O código não muda, porque nunca usa endereço de servidor:

REGRA · UI-00

O front chama a API por caminho relativo (`/api/...`). Verificação: `grep -rn "localhost:3000" src` não encontra nada.

O `App` do template virou um verificador: um botão cujo **handler de evento** chama `fetch("/health")` e guarda o resultado em estado. Buscar dados ao abrir a tela pede `useEffect`, e há bons motivos para esperar até o capítulo 7. A sessão também passa pelo proxy: este roteiro entra como Ana pela tela e consulta quem está logado.

medicoes/cap02_cookie.py · Medido

```
chromium 141.0.7390.37 · http://localhost:5173/
tela: API ok (via proxy /health)
POST /api/auth/login  -> 200
cookie __Host-sessao: httpOnly=True secure=True
sameSite=Lax
document.cookie       -> ''
GET  /api/auth/me     -> 200 ana@hospital.dev
```

O cookie chegou `HttpOnly`, `Secure` e `SameSite=Lax`, o JavaScript da página não o lê, e mesmo assim o pedido seguinte saiu autenticado: o navegador anexa o cookie sozinho. O capítulo 15 constrói a sessão da tela sobre isso; o porquê de cada atributo está no *Crachá & Porta*, capítulo 5.

Para inspecionar, a extensão React Developer Tools acrescenta as abas **Components** (árvore, props e estado) e **Profiler** (quem renderizou e quanto levou). O Vite entrega *source maps*: os *breakpoints* param no `.tsx` original.

2.5

Git e o contexto do agente

O primeiro commit vem antes da primeira feature, com quatro arquivos que o agente lê antes de qualquer tarefa:

ARQUIVO	O QUE GUARDA
`AGENTS.md`	Comandos, como o projeto funciona e regras; formato que Codex e agy leem
`CLAUDE.md`	Uma linha, `@AGENTS.md`: o Claude Code importa o mesmo conteúdo
`INVARIANTES.md`	Regras com verificação; começa com a UI-00
`IA.md`	Cada tarefa feita com agente: pedido, verificação, o que foi recusado

PONTE · DEV DE IA, CAPS. 3 A 5

Por que o contexto deve ser curto e como um invariante ganha verificação executável: o mapa dos cinco arquivos (cap. 3), `CLAUDE.md` e `AGENTS.md` (cap. 4) e `INVARIANTES.md` (cap. 5).

2.6

Com agentes de IA

A primeira tarefa num projeto novo é conferir se o agente entendeu o projeto, em modo de plano, sem editar nada. O conteúdo do prompt é o mesmo nas três ferramentas; muda como você planeja, revisa e roda sem interação:

PASSO	CLAUDE CODE	CODEX · AGY
Planejar	Shift+Tab (Plan Mode)	`/plan` · modo de plano
Revisar	`/code-review`	`/review` · subagente
Sem interação	`claude -p`	`codex exec` · `agy -p`

PROMPT · AGENTE (MODO DE PLANO) · CAP 2

```
Goal
Explicar o caminho de um clique em
"Verificar API" até a resposta da API.

Context
```

```
@AGENTS.md @vite.config.ts @src/
A API roda em :3000 (outro repositório).

Constraints
- só leitura: não edite nada
- cite arquivo e linha de cada passo
- diga o que muda em produção

Done when
- cita main.tsx, App.tsx, vite.config.ts
- explica por que não há CORS no caminho
- git status continua limpo
```

ARMADILHA DE IA

Diante de um erro de CORS, o agente costuma propor `fetch("http://localhost:3000/...")` e liberar `origin: "*"` na API. As duas mudanças funcionam e estão erradas: a primeira quebra a UI-00 e a produção, a segunda abre a API para qualquer site. O `AGENTS.md` já proíbe mexer na API e no Vite. Comandos de agente mudam rápido; confira na versão instalada.

Exercícios

ESSENCIAL Gere o projeto, configure o proxy e faça o botão mostrar **ok**. Pare a API e clique de novo: o que aparece, e por quê?

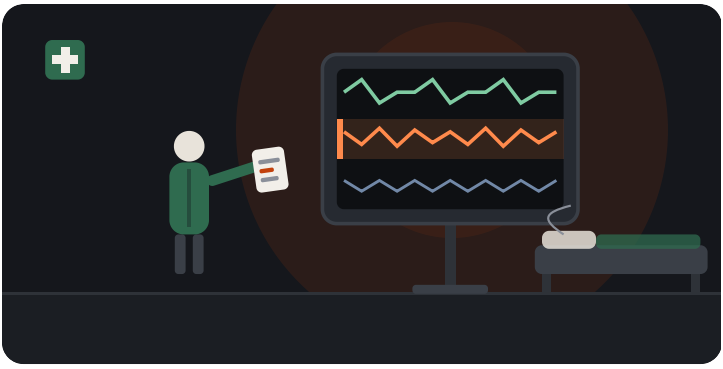
INTERMEDIÁRIO Introduza um erro de tipo no `App.tsx`. Rode `npm run dev` e depois `npm run gate` e explique a diferença.

DESAFIO

Rode `npm run build` e `npm run preview`: o botão funciona. Descubra na documentação do Vite por quê, e escreva a configuração que um servidor de produção (nginx, por exemplo) precisaria para a UI-00 continuar valendo.

Régua: prepara 34 e 35. **Invariantes:** UI-00. **Tag:** `cap02`. **Prompts:** `prompts/cap02.md`.

Fontes. Vite 8 (mar. 2026), *server.proxy*, *preview.proxy*; react.dev: *createRoot*, *StrictMode*; React Router v8; TypeScript 7 (InfoQ). Template de 21 out. 2026. Saídas em `saidas/c2-*.txt`.



PARTE I · A MUDANÇA DE PARADIGMA

03

Como o React funciona por dentro

Gatilho, render e commit, identidade e keys, provados com o motor ligado.

Quarta, 10h. A enfermeira anota "ver febre" no campo de João Lima e dá alta a Maria Souza. A anotação aparece na linha de Ana Clara Reis; João fica com o campo vazio.

O código da alta está certo. O problema está numa linha que parece irrelevante, e para achá-lo é preciso saber o que o React faz entre a sua função devolver a tela e o navegador mostrá-la.

3.1

O projeto do capítulo

Na tag `cap03`, a Ala B virou componentes com JSX e TypeScript, em `src/features/pacientes`. O cartão de cada paciente:

src/features/pacientes/CartaoPaciente.tsx `cap03`

```tsx
import type { Paciente } from "./tipos.ts";

interface Props {
  paciente: Paciente;
  onAlta: (id: string) => void;
}

export function CartaoPaciente(
  { paciente, onAlta }: Props,
) {
  const { id, nome, leito } = paciente;
  return (
    <li>
      <b>{nome}</b> {leito}
      <input
        aria-label={`Anotação de ${nome}`}
        placeholder="anotação"
      />
      <button onClick={() => onAlta(id)}>
        Alta
      </button>
    </li>
  );
}
```

A `Ala` guarda `busca` e `pacientes` em estado e calcula o resto, exatamente como a versão 3 do capítulo

1, agora em JSX.

3.2

JSX é açúcar

O navegador não entende JSX. Antes de chegar a ele, cada tag vira uma chamada de função, como o `h` do capítulo 1. É isto que o Vite entrega quando o navegador pede o cartão:

servidor do Vite · Medido

```
GET /src/features/pacientes/CartaoPaciente.tsx
(trecho; metadados de depuração viraram …)

export function CartaoPaciente({ paciente, onAlta
}) {
  const { id, nome, leito } = paciente;
  return _jsxDEV("li", { children: [
    _jsxDEV("b", { children: nome }, …),
    " ",
    leito,
    _jsxDEV("input", {
      "aria-label": `Anotação de ${nome}`,
      placeholder: "anotação"
    }, …),
    _jsxDEV("button", {
      onClick: () => onAlta(id),
      children: "Alta"
    }, …)
  ] }, …);
}
```

Em desenvolvimento a função se chama `jsxDEV` e carrega arquivo e linha de cada tag para as mensagens

de erro; em produção, `jsx`. As chaves do JSX viram expressões comuns, os filhos viram a prop `children`, e o resultado é um **elemento**: um objeto que *descreve* um `<li>`. Nenhum nó do DOM foi criado.

3.3

Gatilho, render, commit



Fig. 3.1 — As etapas de toda atualização. O **gatilho** é a montagem inicial ou um `set`. No **render**, o React chama os componentes e recebe a árvore nova. No **commit**, compara com a anterior e aplica no DOM só o que mudou. A pintura já é do navegador; os efeitos são o capítulo 7.

H+ **A passagem de plantão**

Na troca de turno, a enfermeira compara o quadro de ontem com a situação de hoje e anota só o que mudou. Olhar a situação é o render; anotar a diferença é o commit.

ONDE A ANALOGIA PARA

O plantão tem hora marcada; o render só acontece com gatilho e pode terminar sem diferença nenhuma. O React pode até descartar um render pela metade para atender algo mais urgente. O commit, nunca: é aplicado inteiro, de uma vez.

3.4

Renderizar não é mexer no DOM

Para separar as duas coisas, o projeto tem o `Renderometro`, construído sobre o `<Profiler>` do React: a cada commit em que algo dentro dele renderizou, o React chama `onRender` com o nome, a fase e o tempo.

src/lab/Renderometro.tsx cap03

```
export function Renderometro(
  { id, children }: Props,
) {
  return (
    <Profiler
      id={id}
      onRender={(quem, fase, ms) => {
```

```
        const linha = `${quem} · ${fase}`
          + ` · ${ms.toFixed(2)} ms`;
        registros.push(linha);
        console.log(`[render] ${linha}`);
      }}
    >
      {children}
    </Profiler>
  );
}
```

A lista e cada cartão estão dentro de um `Renderometro`. Um roteiro no Chromium cruza o que ele anotou com o que um `MutationObserver` viu no DOM:

medicoes/cap03.py · Medido

```
digitar "o" na busca
  Maria Souza     update
  João Lima       update
  Pedro Alves     update
  Raimundo Nonato update
  Lista           update
  DOM: +0 -8 ~1
…
anotar em João Lima
  DOM: +0 -0 ~0
```

Uma tecla na busca fez a lista e os quatro cartões visíveis renderizarem de novo. No DOM, porém, só saíram as duas linhas que deixaram de bater, oito elementos, e mudou o `value` da busca. Os cartões foram *calculados* de novo, deram o mesmo resultado, e o commit não os tocou.

Digitar na anotação de João não provocou render nenhum: aquele campo não tem estado no React, o texto vive no `<input>` do DOM. Guarde isso para a seção 3.6.

3.5

O "Virtual DOM", com precisão

"Virtual DOM" é o apelido da árvore de elementos que cada render devolve. Ele **não é uma cópia do DOM mantida em sincronia**: é a descrição produzida por um render, comparada com a do render anterior. E **não é sempre mais rápido** que mexer no DOM à mão: no capítulo 1, imperativo e React fizeram o mesmo trabalho no DOM, e o React ainda executou componentes. O que ele entrega é correção sem esforço manual.

A comparação, a **reconciliação**, segue regras simples:

NA MESMA POSIÇÃO	O QUE O REACT FAZ
Mesmo tipo	Mantém o nó e o estado; atualiza o que mudou nas props
Tipo diferente	Desmonta o que havia, com o estado, e monta do zero
Em listas, com `key`	Casa os itens pela key, não pela posição

3.6

Identidade: o estado mora na posição

A regra da documentação é curta: **o React mantém o estado de um componente enquanto ele for renderizado na mesma posição da árvore**. Ser "o mesmo", para o React, é estar no mesmo lugar, com o mesmo tipo e, em listas, a mesma key. Na tag `cap03-antes`, a lista usava o índice como key:

```
src/features/pacientes/ListaPacientes.tsx                cap03-antes → cap03

…
      <ul>
-       {pacientes.map((p, i) => (
-         <Renderometro key={i} id={p.nome}>
+       {pacientes.map((p) => (
+         <Renderometro key={p.id} id={p.nome}>
            <CartaoPaciente
```



Fig. 3.2 — Com key por índice, o React vê que a última posição sumiu e que as outras só trocaram de props. O `<input>` da posição 1 fica no DOM, com o texto, mostrando agora Ana Clara.

O texto nunca esteve no React; estava no nó. Com key por índice, os nós são reaproveitados pela posição e o texto fica preso a eles. Com o `id`, o React sabe que `p1` saiu e que `p2` continua `p2`. O teste descreve o que a enfermeira faz, sem olhar para dentro do componente:

src/features/pacientes/Ala.test.tsx cap03

```
test("anotação acompanha o paciente",
  async () => {
    render(<Ala />);
    await userEvent.type(
      nota("João Lima"), "ver febre");
    await darAlta("Maria Souza");
    expect(nota("João Lima").value)
      .toBe("ver febre");
  });
```

CAP03-ANTES · KEY={I}

```
× anotação acompanha o paciente
→ expected '' to be 'ver febre'
```

CAP03 · KEY={P.ID}

```
✓ anotação acompanha o paciente
✓ alto risco cai na alta
```

É a **prova de revert** da série: o teste falha sem a correção e passa com ela. As saídas completas estão em `saidas/c3-key-*.txt`. Trocar a key de propósito também serve para zerar um componente; o capítulo 6 usa isso.

REGRA · UI-01

Lista usa a key do domínio (o `id`), nunca o índice. Verificação: o teste "anotação acompanha o paciente".

3.7

Montar, atualizar, desmontar

Montar é aparecer numa posição: nós criados, estado no valor inicial. **Atualizar** é renderizar de novo no mesmo lugar. **Desmontar** é sair da árvore, levando nós e estado. Na medição completa, Ana Clara e Luzia voltam como `mount` quando a busca é limpa: tinham sido desmontadas.

Para isso funcionar, o render precisa ser **puro**: as mesmas props e o mesmo estado produzem a mesma tela, e renderizar não altera nada fora do componente. O `StrictMode` testa a promessa chamando cada componente duas vezes em desenvolvimento, e é por isso que o `Renderometro` conta commits pelo `Profiler` em vez de somar uma variável no corpo do componente. Internamente, o render é dividido em unidades de trabalho, as *fibers*, o que permite pausá-lo; recursos como `startTransition` usam isso e aparecem quando forem úteis.

3.8

Com agentes de IA

ARMADILHA DE IA

Agentes geram `key={index}` com frequência, porque é o exemplo mais curto, e às vezes `key={Math.random()}`, que é pior: cada render remonta tudo e apaga todo estado da lista. Nenhum dos dois quebra o build. A UI-01 e o teste pegam ambos.

Para um bug como o da abertura, o primeiro prompt pede diagnóstico com evidência, não correção:

PROMPT · AGENTE · CAP 3 · DIAGNÓSTICO

```
Goal
Descobrir por que a anotação de um
paciente aparece em outro depois de
uma alta, e provar a causa.

Context
@src/features/pacientes/ @INVARIANTES.md
Passos: anotar em João Lima, dar alta a
Maria Souza; a nota vai para Ana Clara.

Constraints
- antes, um teste que reproduz o defeito
  pelo que o usuário vê, falhando
- depois, a correção mínima
- sem dependência nova

Done when
- npm run gate verde com a correção
- o teste novo FALHA se a correção
```

```
  for revertida
- a causa aponta arquivo e linha
```

> **PONTE · DEV DE IA, CAPS. 9 E 13**
>
> O "falha se revertido" vem do capítulo 9: agentes escrevem com facilidade testes que passam antes e depois da mudança. A revisão em contexto novo, de preferência com outra ferramenta, é o capítulo 13; o prompt dela está em `prompts/cap03.md`.

Exercícios

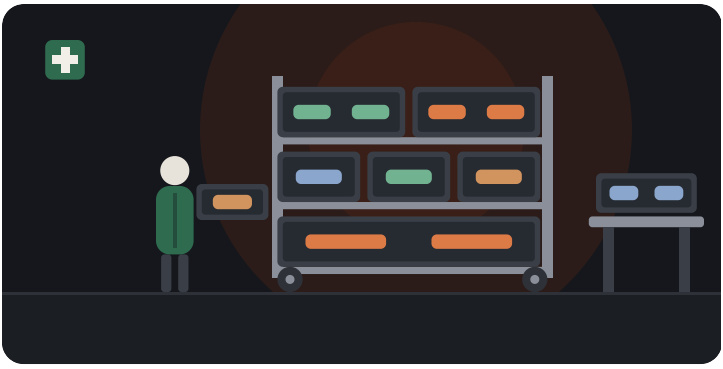
ESSENCIAL Rode `npm test` em `cap03-antes` e em `cap03`. Explique a diferença em três frases, usando "posição" e "key".

INTERMEDIÁRIO Preveja quais componentes renderizam ao dar alta ao último paciente. Rode com o `Renderometro` e compare. Por que a previsão para key por índice seria diferente?

DESAFIO Troque a key por `Math.random()`, meça uma tecla na busca com `medicoes/cap03.py` e explique o número.

Régua: 02, 03, 04. **Invariantes:** UI-01. **Tags:** `cap03-antes`, `cap03`. **Prompts:** `prompts/cap03.md`.

Fontes. react.dev: *Render and Commit*, *Preserving and Resetting State*, *Rendering Lists*, *Keeping Components Pure*, *<Profiler>*, *<StrictMode>*. Saídas em `saidas/c3-*.txt`.



PARTE II · OS FUNDAMENTOS

04

Componentes, JSX e composição

Peças pequenas que se encaixam. Nem toda repetição pede um componente, e nenhum componente nasce dentro de outro.

Quinta, 15h. A tela da Ala B ganhou resumo por risco, um selo colorido em cada paciente e um aviso para busca sem resultado. Tudo foi parar dentro de `Ala`. Desde então, a cada letra digitada na busca, as anotações somem.

O código novo não tem erro de lógica. Tem um erro de *forma*: um componente foi definido no lugar errado. Este capítulo trata de como dividir uma tela em componentes, e dos dois jeitos de errar nisso.

4.1

Um componente é uma função

Um componente é uma função que recebe um objeto, as **props**, e devolve JSX. Poucas regras sustentam tudo:

REGRA	EXEMPLO
Nome com maiúscula; minúscula é tag HTML	`<CartaoPaciente />` × `<li>`
Uma raiz, ou um fragmento	`<>…</>` agrupa sem criar nó
Atributos em camelCase	`className`, `onClick`, `aria-label` fica igual
Chaves para expressões	`{pacientes.length}`; `if` não cabe, ternário cabe
Toda tag fecha	`<input />`, ` `

4.2

O componente que cresceu demais

Na tag `cap04-antes`, a `Ala` tem 74 linhas e faz tudo. No meio dela, para "organizar", alguém extraiu a linha da lista para um componente `Linha`, escrito *dentro* de `Ala`:

Ala.tsx · Linha, dentro de Ala — cap04-antes

```
function Linha({ p }: { p: Paciente }) {
  return (
```

```
      <li>
        <b>{p.nome}</b> {p.leito}
        <span data-risco={p.risco}>
          {p.risco}
        </span>
        <input
          aria-label={`Anotação de ${p.nome}`}
          placeholder="anotação"
        />
        <button onClick={() => darAlta(p.id)}>
          Alta
        </button>
      </li>
    );
  }
```

Parece inofensivo, e o código usa `key={p.id}` corretamente. O problema é a regra do capítulo 3: **tipo diferente na mesma posição, o React desmonta e monta de novo**. A cada render de `Ala`, a função `Linha` é criada outra vez. É uma função nova, logo um tipo novo. O React não tem como saber que é "a mesma `Linha`": desmonta todas as linhas, com o que havia nelas, e monta outras.

medicoes/cap04.py · Medido

```
== cap04-antes

digitar "o" na busca
  Maria Souza     update
  João Lima       update
  Pedro Alves     update
  Raimundo Nonato update
  DOM: +20 -30 ~1
  anotação de João: ''
```

```
== cap04

digitar "o" na busca
  Maria Souza     update
  João Lima       update
  Pedro Alves     update
  Raimundo Nonato update
  DOM: +0 -10 ~1
  anotação de João: 'ver febre'
```

Na mesma tecla, `cap04-antes` recriou 20 elementos e removeu 30: as quatro linhas que ficaram foram reconstruídas e a anotação de João foi junto. Em `cap04` saíram só os 10 elementos das duas linhas filtradas. Repare que o `Renderometro` diz `update` nos dois casos: o `Profiler` está do lado de fora de `Linha`, na mesma posição, e não vê a troca de tipo lá dentro. Quem mostra a verdade é o DOM.

CAP04-ANTES · LINHA DENTRO DE ALA

`× anotação acompanha o paciente`
`× anotação sobrevive à busca`

CAP04 · COMPONENTES NO MÓDULO

`✓ os quatro testes`, inclusive os dois acima, sem mudar uma linha do teste

O lint do template não percebe o problema. O oxlint tem a regra, mas ela não vem ligada:

```
terminal · Medido

$ npx oxlint            # regras do template
(nenhuma saída) · exit 0

$ npx oxlint -D react/no-unstable-nested-
components
src/features/pacientes/Ala.tsx:25:3
  error react(no-unstable-nested-components)
  Do not define components during render.
  help: Move this component definition out of the
parent component `Ala` and pass data as props.
exit 1
```

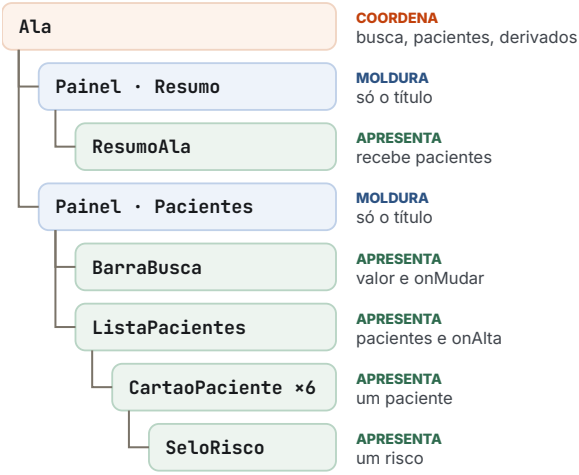
REGRA · UI-02

Componente nunca é definido dentro de outro. Verificação: `react/no-unstable-nested-components` ligada no `.oxlintrc.json`, que o `npm run gate` roda.

4.3

Composição

A correção, na tag `cap04`, não é só tirar `Linha` de dentro de `Ala`. É dividir a tela pelo que cada pedaço *é* para quem usa: o resumo, a busca, a lista, o cartão, o selo.



Quem coordena guarda estado. Quem apresenta recebe props. A moldura não sabe o que carrega.

Fig. 4.1 — A árvore da Ala na tag `cap04`. Três papéis: quem **coordena** guarda estado e compõe; quem **apresenta** recebe props e mostra; a **moldura** recebe conteúdo pronto.

Agora a `Ala` só coordena. O que ela devolve se lê como a própria tela:

Ala.tsx · composição cap04

```
return (
  <>
    <Painel titulo="Resumo">
      <ResumoAla pacientes={pacientes} />
    </Painel>
    <Painel titulo="Pacientes">
      <BarraBusca
        valor={busca}
        onMudar={setBusca}
      />
```

```
        {visiveis.length === 0 ? (
          <p>Nenhum paciente encontrado.</p>
        ) : (
          <ListaPacientes
            pacientes={visiveis}
            onAlta={darAlta}
          />
        )}
      </Painel>
    </>
  );
```

O `Painel` é a moldura. Ele recebe o conteúdo pela prop especial `children`, tudo o que vem entre a tag de abertura e a de fechamento, e não sabe o que está mostrando:

src/compartilhado/Painel.tsx cap04

```
// Moldura com título. Não sabe o que mostra:
// quem a usa decide, pelos children.
export function Painel(
  { titulo, children }: Props,
) {
  return (
    <section className="painel">
      <h2>{titulo}</h2>
      {children}
    </section>
  );
}
```

H+ **As bandejas do carrinho**

O carrinho de procedimentos não tem um compartimento para cada combinação de material. Tem bandejas padronizadas que se encaixam de jeitos diferentes. O `Painel` é o encaixe; o que vai dentro muda a cada uso.

ONDE A ANALOGIA PARA

Bandeja física tem tamanho fixo. A composição do React aninha sem limite: um `Painel` dentro de outro, uma lista dentro de um painel. O custo de aninhar é só de leitura, não de desempenho.

4.4

Composição, duplicação e abstração prematura

Dividir demais também é erro. O sintoma é o componente genérico que tenta servir a todos os casos:

ABSTRAÇÃO PREMATURA

`<Cartao tipo="paciente" mostrarSelo mostrarBotaoAlta titulo={…} subtitulo={…} … />`: dez props booleanas, um `if` para cada combinação, e ninguém sabe qual combinação a tela usa.

COMPOSIÇÃO

Componentes de domínio pequenos (`CartaoPaciente`, `SeloRisco`) e uma moldura que recebe `children`. Cada um tem um motivo para mudar.

O projeto adota três critérios, escritos em `docs/UI.md` para que o agente os siga também:

DOCS/UI.MD · QUANDO EXTRAIR UM COMPONENTE

- Um trecho com nome próprio na conversa do usuário ("o resumo").
- Um trecho repetido pela terceira vez. Na segunda, copie e espere.
- Nunca para "deixar genérico" antes de existir o segundo uso real.

PONTE · DEV DE IA, CAP. 6

O `docs/UI.md` também fixa um componente por arquivo, nomes em PascalCase e props numa `interface Props`. O Dev de IA explica por que esse arquivo reduz retrabalho e como mantê-lo curto.

4.5

Com agentes de IA

ARMADILHA DE IA

Os agentes erram para os dois lados. Pedidos de "mais uma funcionalidade" sem contexto tendem a crescer o componente que já existe até virar um componente-deus, muitas vezes com um componente auxiliar definido lá dentro, exatamente o defeito deste capítulo. Pedidos de "deixe reutilizável" tendem a produzir o cartão genérico com dez props. O `docs/UI.md` e a UI-02 cortam os dois.

PROMPT · AGENTE · CAP 4

```
Goal
Dividir a Ala em componentes de domínio,
sem mudar o comportamento.

Context
@src/features/pacientes/ @docs/UI.md
@INVARIANTES.md

Constraints
- um componente por arquivo, nomes de
  domínio, Props tipadas
- Ala só guarda estado, deriva e compõe
- nenhum componente dentro de outro
- moldura genérica só em compartilhado/
- não altere Ala.test.tsx

Done when
- npm run gate verde, com a regra
  no-unstable-nested-components ligada
```

```
- Ala.test.tsx passa sem nenhuma mudança
- git diff não toca Ala.test.tsx
```

O último critério é o que transforma "refatorar" em algo verificável: se o teste precisou mudar, o comportamento mudou. A revisão e as variantes estão em `prompts/cap04.md`.

Exercícios

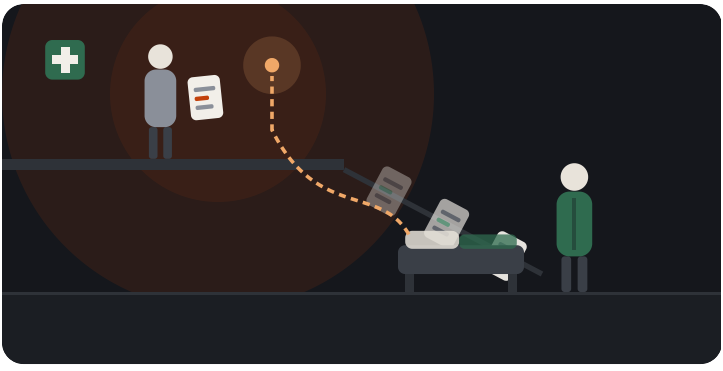
ESSENCIAL Em `cap04-antes`, mova `Linha` para fora de `Ala` sem mais nenhuma mudança. Quais props ela passa a precisar? Rode os testes.

INTERMEDIÁRIO Use o `Painel` para criar um terceiro quadro, "Avisos", com uma lista fixa de dois avisos. Não mude o `Painel`.

DESAFIO Peça a um agente um "cartão reutilizável para pacientes e profissionais". Avalie a resposta pelos critérios do `docs/UI.md` e registre no `IA.md` o que você recusou.

Régua: 05. **Invariantes:** UI-02. **Tags:** `cap04-antes`, `cap04`. **Prompts:** `prompts/cap04.md`.

Fontes. react.dev: *Your First Component*, *Writing Markup with JSX*, *Passing Props to a Component* (children), *Preserving and Resetting State* (não aninhar definições). oxlint: *react/no-unstable-nested-components*. Saídas em `saidas/c4-*.txt`, Chromium 141.



PARTE II · OS FUNDAMENTOS

05

Props, eventos e fluxo de dados

Dados descem como props, eventos sobem como chamadas. Nenhum componente altera o que recebeu.

Sexta, 8h. A enfermeira seleciona João Lima e clica em "Prioridade". Nada acontece. Ela digita uma letra na busca, e a estrela de prioridade aparece em João, sem que ninguém tenha clicado de novo.

Uma mudança que só aparece quando outra coisa acontece é a assinatura de um erro específico. Antes de chegar nele, o capítulo monta a seleção de pacientes e

o painel de detalhe, e com eles o fluxo de dados do React.

5.1

Props: o contrato do componente

As props de um componente são o seu contrato com quem o usa. Em TypeScript, o contrato fica explícito numa `interface`:

CartaoPaciente.tsx · contrato cap05

```tsx
interface Props {
  paciente: Paciente;
  selecionado: boolean;
  onSelecionar: (id: string) => void;
  onPrioridade: (id: string) => void;
  onAlta: (id: string) => void;
}

export function CartaoPaciente({
  paciente, selecionado,
  onSelecionar, onPrioridade, onAlta,
}: Props) {
  const { id, nome, leito, risco } = paciente;
  const atual = selecionado || undefined;
```

- **Obrigatória** é a prop sem `?`: esquecê-la é erro de tipo, não bug em produção.
- **Opcional** leva `?` e pode chegar `undefined`, como `selecionadoId?: string` na lista. Valor padrão vai na desestruturação: `{ risco = "baixo" }`.
- Funções também são props. Por convenção, a prop se chama `onAlgo` e a função de quem a passa tem

um nome de ação, como `darAlta`.

Props são **somente leitura**. O componente que recebe `paciente` não altera `paciente.prioridade`: ele avisa quem é dono do dado, chamando `onPrioridade(id)`.

5.2

Levantar o estado

A lista precisa saber qual paciente está selecionado, para destacá-lo, e o painel de detalhe precisa mostrá-lo. Quando dois componentes dependem do mesmo dado, o estado sobe para o ancestral comum mais próximo, aqui a `Ala`:

Ala.tsx · estado e derivados cap05

```
export function Ala() {
  const [busca, setBusca] = useState("");
  const [pacientes, setPacientes] =
    useState(ALA_B);
  const [selecionadoId, setSelecionadoId] =
    useState<string>();

  const t = busca.toLowerCase();
  const visiveis = pacientes.filter((p) =>
    p.nome.toLowerCase().includes(t));
  const selecionado = pacientes
    .find((p) => p.id === selecionadoId);
```

O estado guarda o **id** do selecionado, não uma cópia do objeto. O paciente selecionado é calculado a cada render a partir da lista. Se a `Ala` guardasse uma cópia,

a prioridade mudaria na lista e a cópia no painel continuaria velha: dois estados para a mesma verdade.

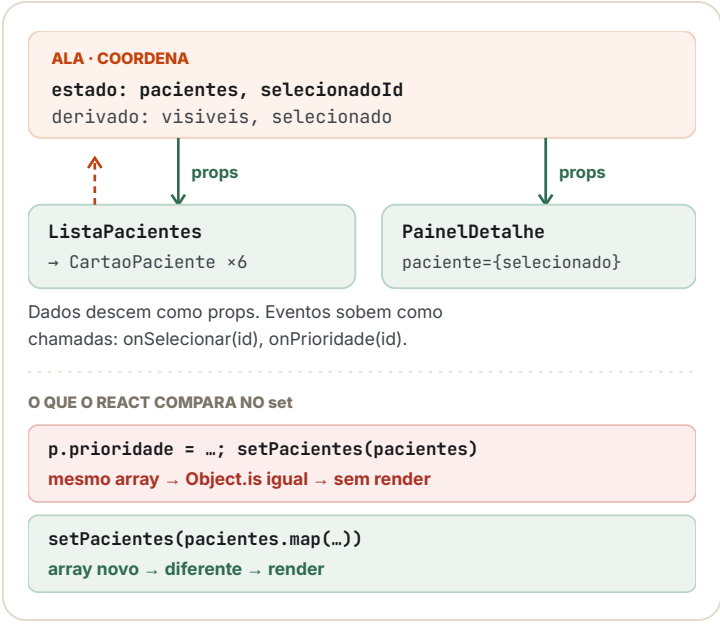


Fig. 5.1 — O fluxo de mão única. A `Ala` é dona do estado; lista e detalhe recebem props. O cartão não altera nada: chama `onSelecionar` ou `onPrioridade`, e a `Ala` decide. Embaixo, o que o React compara quando você chama `set`.

H+

A prescrição e a campainha

A prescrição desce do médico para a enfermagem, e quem a recebe não a reescreve. Quando o paciente precisa de algo, ele não vai ao posto mudar a prescrição: toca a campainha, e quem é responsável decide. Props descem; eventos sobem.

ONDE A ANALOGIA PARA

A campainha só avisa que algo aconteceu. O callback do React leva dados (`onPrioridade(id)`) e roda na hora, dentro do componente dono do estado.

A lista não usa os eventos; ela só os repassa ao cartão. O espalhamento de props (`...acoes`) evita repetir as três linhas:

src/features/pacientes/ListaPacientes.tsx cap05

```
export function ListaPacientes({
  pacientes, selecionadoId, ...acoes
}: Props) {
  return (
    <ul>
      {pacientes.map((p) => (
        <Renderometro key={p.id} id={p.nome}>
          <CartaoPaciente
            paciente={p}
            selecionado={
              p.id === selecionadoId}
            {...acoes}
          />
        </Renderometro>
      ))}
    </ul>
  );
}
```

Repassar uma prop por um componente que não a usa é normal em um ou dois níveis. Quando são cinco, o nome disso é *prop drilling*, e o capítulo 11 trata das saídas.

5.3

Renderização condicional

O painel de detalhe tem dois estados visuais, e o jeito mais legível de expressá-los é um retorno antecipado:

src/features/pacientes/PainelDetalhe.tsx cap05

```
…
export function PainelDetalhe(
  { paciente }: Props,
) {
  if (!paciente) {
    return <p>Selecione um paciente.</p>;
  }
…
```

ARMADILHA DO && COM NÚMERO

`{visiveis.length && <ListaPacientes … />}` mostra **0** na tela quando a lista está vazia: o React renderiza números, e `0 && x` vale `0`.

CONDIÇÃO BOOLEANA

`{visiveis.length > 0 && …}`, ou o ternário com o estado vazio, como a `Ala` faz.

5.4

O erro clássico: mudar o objeto no lugar

Na tag `cap05-antes`, a prioridade era alterada assim, e a correção de `cap05` cria um array novo:

src/features/pacientes/Ala.tsx — `cap05-antes → cap05`

```
…
   function alternarPrioridade(id: string) {
-    const p = pacientes
-      .find((x) => x.id === id)!;
-    p.prioridade = !p.prioridade;
-    setPacientes(pacientes);
+    setPacientes(pacientes.map((p) =>
+      p.id === id
+        ? { ...p, prioridade: !p.prioridade }
+        : p));
   }
```

A versão antiga altera o objeto que já está no estado e chama `setPacientes` com **o mesmo array**. O React compara o valor novo com o anterior por referência, com `Object.is`. É o mesmo array, então nada mudou, e nenhum render acontece. O objeto, porém, foi alterado de verdade, e aparece no próximo render que qualquer outra coisa provocar:

medicoes/cap05.py · Medido

```
== cap05-antes
clique em Prioridade: 0 render(s), ★ na tela: não
tecla na busca:       4 render(s), ★ na tela: sim

== cap05
clique em Prioridade: 6 render(s), ★ na tela: sim
tecla na busca:       4 render(s), ★ na tela: sim
```

Zero renders no clique; a estrela só aparece quando a busca provoca um render por outro motivo. A versão corrigida cria um array novo e um objeto novo só para o paciente alterado (`{ ...p, prioridade }`). Os outros cinco objetos continuam os mesmos, o que importa no capítulo 16, quando a memoização entrar.

CAP05-ANTES

`× prioridade no cartão e no detalhe`

CAP05

`✓ os seis testes`

REGRA · UI-03

Estado nunca é alterado no lugar: toda atualização cria objeto ou array novo. Verificação: o teste "prioridade no cartão e no detalhe".

5.5

Com agentes de IA

ARMADILHA DE IA

Mutação escondida em métodos que parecem leitura é o caso mais comum em código gerado: `pacientes.sort(...)` e `reverse()` alteram o array original, e `push` e `splice` também. Para ordenar sem mutar, use `toSorted()`, do ES2023, ou ordene uma cópia.

PROMPT · AGENTE · CAP 5

```
Goal
Ordenar a lista da Ala por risco (alto
primeiro), com um botão que liga e
desliga a ordenação.

Context
@src/features/pacientes/ @INVARIANTES.md
@docs/UI.md

Constraints
- a ordem é derivada no render, nunca
  guardada como outra lista em estado
- nada de sort, reverse, push ou splice
  sobre estado (UI-03)
- o botão usa aria-pressed

Done when
- npm run gate verde
- teste novo: ligar, desligar e conferir
  a ordem original de volta
- os seis testes atuais passam sem mudar
```

Exercícios

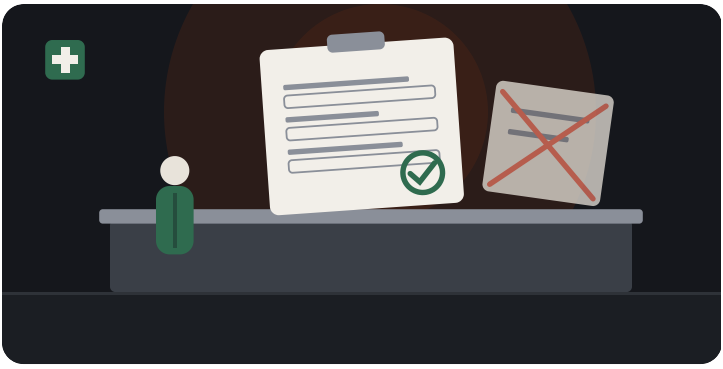
ESSENCIAL Em `cap05-antes`, troque `setPacientes(pacientes)` por `setPacientes([...pacientes])` e rode os testes. Passa? Explique por que essa correção resolve o sintoma e mesmo assim viola a UI-03.

INTERMEDIÁRIO Faça o painel de detalhe guardar uma cópia do paciente em `useState`. Marque a prioridade e descreva o que fica dessincronizado.

DESAFIO Implemente a ordenação por risco do prompt sem usar agente. Depois rode o prompt num agente e compare as duas versões com o V.E.R.M.

Régua: 06; prepara 08. **Invariantes:** UI-03. **Tags:** `cap05-antes`, `cap05`. **Prompts:** `prompts/cap05.md`.

Fontes. react.dev: *Passing Props to a Component*, *Responding to Events*, *Sharing State Between Components*, *Conditional Rendering*, *Updating Arrays in State*, *useState* (comparação com `Object.is`). MDN: *Array.prototype.toSorted*. Saídas em `saidas/c5-*.txt`, Chromium 141.



PARTE II · OS FUNDAMENTOS

06

Estado, formulários e interações

Guarde o mínimo, calcule o resto, e trate cada envio como algo que pode chegar duas vezes.

Segunda, 7h40. A rede do pronto-socorro está lenta. A recepcionista admite Rosa Amaral, nada parece acontecer, e ela clica de novo. Rosa ganha dois prontuários com o mesmo CNS, e o exame da tarde vai para um deles.

O formulário estava certo: validava, enviava e mostrava o resultado. Faltava tratar o tempo entre o clique e a resposta. Este capítulo monta os dois formulários do Mini-Prontuário, o de entrada e o de admissão, e

termina com a defesa contra esse erro, medida no navegador.

6.1

O estado é uma foto

`useState` devolve o valor *deste render* e uma função para pedir o próximo. Chamar `setX` não muda `x` na linha seguinte; agenda um render novo, em que `x` terá o valor novo.

Por isso `setTotal(total + 1)` três vezes seguidas soma 1: as três chamadas leem o mesmo `total` da foto. Com a **atualização funcional**, `setTotal((t) => t + 1)`, cada chamada recebe o resultado da anterior e a soma dá 3. Use essa forma sempre que o próximo valor depender do anterior.

6.2

Guardar ou calcular

Antes de criar um estado, faça a pergunta do capítulo 5: **dá para calcular?** Total de pacientes, lista filtrada, paciente selecionado e "o formulário é válido?" são **derivados**: calculados a cada render, a partir de outro estado ou de props. Estado guardado em dobro precisa ser sincronizado à mão, e a sincronização é onde nascem os bugs. O capítulo 7 mostra o efeito que tenta fazer essa sincronização e por que ele é o erro mais comum de React.

REGRA PRÁTICA

Estado é o que o usuário fez ou o que o servidor disse. Todo o resto é derivado.

6.3

Controlado e não controlado

Um campo pode guardar o valor em dois lugares. No **controlado**, o valor mora no estado do React e cada tecla passa por ele. É o formulário de entrada:

FormLogin.tsx · controlado — cap06

```
…
// Formulário controlado: cada tecla passa
// pelo estado do React.
export function FormLogin() {
  const [email, setEmail] = useState("");
  const [senha, setSenha] = useState("");
  const [estado, setEstado] =
    useState<Estado>({ tipo: "ocioso" });

  async function entrar(
    e: FormEvent<HTMLFormElement>,
  ) {
    e.preventDefault();
    setEstado({ tipo: "enviando" });
…
```

No **não controlado**, o valor mora no próprio `<input>`, e o código só o lê no envio, com `FormData`. É o formulário de admissão, que usa `name` nos campos e `defaultValue` para devolver o que foi digitado quando

há erro. Use controlado quando a tela reage a cada tecla (busca, máscara, botão que só habilita com o campo cheio); não controlado quando o valor só importa no envio, o caso da maioria dos cadastros.

6.4

Os estados de um envio

Todo envio passa pelos mesmos momentos, e a tela precisa de uma resposta para cada um. Em vez de quatro booleanos (`enviando`, `erro`, `ok`, `invalido`) que podem se contradizer, o resultado é um tipo só, uma **união discriminada** do TypeScript:

src/features/admissao/admitir.ts cap06

```
export type Valores = Record<string, string>;
export type Resultado =
  | { tipo: "ocioso" }
  | { tipo: "invalido"; erros: Valores;
      valores: Valores }
  | { tipo: "erro"; msg: string;
      valores: Valores }
  | { tipo: "ok"; id: string };
```

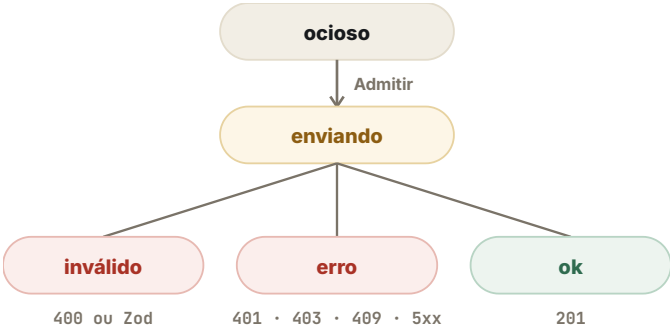


Fig. 6.1 — Os estados de um envio. Com uma união, "ok com mensagem de erro" é impossível de escrever: o TypeScript não deixa.

A função `admitir` valida, envia e traduz a resposta HTTP num desses estados. A validação, em `esquema.ts`, repete as regras do esquema Zod da API (nome com duas letras, CNS com 15 dígitos, data ISO), agora com mensagens para quem preenche.

REGRA · UI-05

A tela valida para ajudar; o servidor valida para decidir. Erro do servidor aparece no campo. Verificação: o teste "mostra o que o servidor recusou", que simula um 400 com `campos: ["cns"]`.

6.5

Actions do React 19

O React 19 trouxe um jeito próprio de tratar envio: passar uma função para `<form action>` e guardar o resultado com `useActionState`. O hook devolve o último resultado, a ação para o formulário e um booleano `pendente`:

FormAdmissao.tsx · action — cap06

```
export function FormAdmissao() {
  const emCurso = useRef(false);
  async function enviar(
    ant: Resultado, dados: FormData,
  ) {
    try {
      return await admitir(ant, dados);
    } finally {
      emCurso.current = false;
    }
  }
  const [res, acao, pendente] =
    useActionState(enviar, INICIAL);
```

A action recebe o resultado anterior e o `FormData`, e o que ela devolve vira o próximo `res`. Não há `preventDefault` nem `setRes` à mão. Um detalhe que surpreende: sempre que a action termina sem lançar erro, o React **reinicia os campos não controlados** para o `defaultValue`. Depois de admitir, os campos ficam vazios; depois de uma recusa, voltam com o que foi digitado, porque o `defaultValue` de cada `Campo` vem de `res.valores`.

6.6

O erro clássico: o envio que chega duas vezes

Na tag `cap06-antes`, o envio era um `onSubmit` comum, e nada impedia o segundo clique. O teste com servidor lento falha:

CAP06-ANTES

```
× não envia duas vezes
→ expected "vi.fn()" to be called 1 times, but got 2 times
```

CAP06

```
✓ não envia duas vezes
✓ dois cliques no mesmo instante
```

A correção óbvia é `disabled={pendente}`, e é a que a documentação mostra. Medimos se basta: um roteiro abre a tela no Chromium contra a API real, deixa a rede 800 ms mais lenta e dá dois cliques no mesmo instante. A linha "sem ref" é o `cap06` com só o `disabled`, aplicado por `medicoes/cap06-so-disabled.patch`.

`medicoes/cap06.py · Medido`

```
tela cap06-antes · api base
  prontuários de Rosa: 2 ['p101', 'p102']
  a tela diz: Paciente p102 admitido.
```

```
tela cap06 sem ref · api base
  prontuários de Rosa: 2 ['p101', 'p102']
  a tela diz: Paciente p102 admitido.

tela cap06 · api base
  prontuários de Rosa: 1 ['p101']
  a tela diz: Paciente p101 admitido.

tela cap06 · api cap06
  prontuários de Rosa: 1 ['p101']
  a tela diz: Paciente p101 admitido.
```

`disabled` **não bastou.** Ele só chega ao DOM no render seguinte, e o segundo clique veio antes. Desabilitar o botão é um aviso visual de que algo está em curso, não uma trava. A trava precisa valer no instante do clique, e para isso serve um `useRef`: uma caixa que guarda um valor entre renders sem provocar render, e que muda na hora.

FormAdmissao.tsx · a trava cap06

```
  // Roda antes da action, no mesmo instante
  // do clique: o ref não espera um render.
  function travar(e: FormEvent) {
    if (emCurso.current) e.preventDefault();
    else emCurso.current = true;
  }
```

O `onSubmit` roda antes da action. O primeiro clique liga a trava; o segundo encontra a trava ligada e cancela o envio. O `finally` da action solta a trava, com sucesso ou com erro.

ATENÇÃO · O TESTE QUE NÃO PERCEBEU

O teste "dois cliques no mesmo instante" falha em `cap06-antes` e passa em `cap06`, mas também passa com só o `disabled`. O jsdom, o navegador simulado do Vitest, não reproduz essa corrida. Por isso a prova deste capítulo é a medição no Chromium. O capítulo 16 volta a esse limite: cada ambiente de teste prova uma coisa.

6.7

A segunda barreira mora no servidor

A trava do ref vale para um formulário, numa aba. Duas abas abertas, um clique em cada, e a tela não tem como saber da outra. Só o servidor vê os dois pedidos. Por isso a API ganhou, na tag `cap06`, uma regra: o CNS identifica a pessoa, e um segundo cadastro com o mesmo CNS responde **409 Conflict**.

`medicoes/cap06.py · Medido`

```
duas abas, mesmo paciente
  api base: 2 prontuário(s)
    aba 2 diz: Paciente p102 admitido.
  api cap06: 1 prontuário(s)
    aba 2 diz: Já existe paciente com este CNS.
```

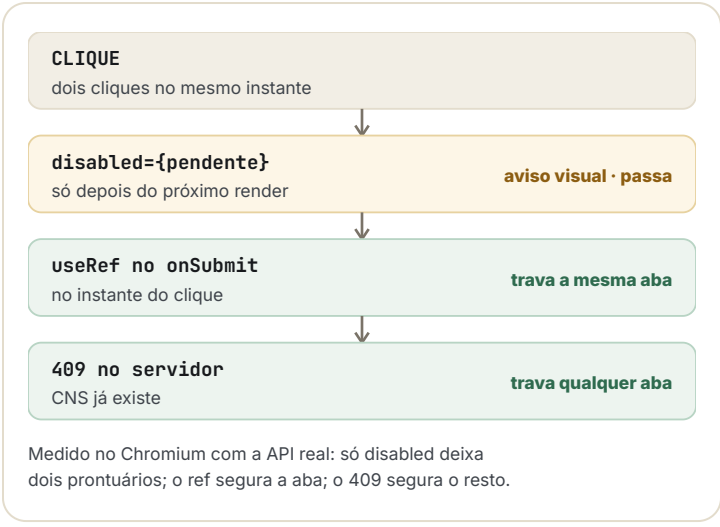


Fig. 6.2 — As barreiras contra o envio duplo, na ordem em que o clique as encontra. A terceira é a única que vale para qualquer cliente.

A função `admitir` trata o 409 à parte e mostra a mensagem do servidor. Antes dessa correção, ele caía em "Falha no servidor. Tente de novo.", uma mensagem que convida exatamente ao clique que gerou o problema.

REGRA · UI-04

Ação que cria registro trava no clique, com `useRef`, e o servidor recusa duplicata com 409. `disabled` é aviso visual, não trava. Verificação: os testes do formulário e `medicoes/cap06.py`.

H+ **O carimbo da admissão**

Na recepção, a ficha carimbada não é carimbada de novo: quem pegou a ficha segura a vez. E o arquivo central confere o CNS antes de abrir uma pasta, porque outra recepção pode estar admitindo a mesma pessoa.

ONDE A ANALOGIA PARA

O carimbo físico é visível na hora. O `disabled` do React não: ele aparece um render depois, e o ref, que trava de verdade, nem aparece na tela.

6.8

Com agentes de IA

ARMADILHA DE IA

Pedido um formulário "que não envie duas vezes", agentes costumam entregar só o `disabled`, porque é o exemplo da documentação. Também é comum a validação ficar só no cliente, e os erros do servidor caírem numa mensagem genérica. A UI-04 e a UI-05 dizem ao agente o que vale.

```
PROMPT · AGENTE · CAP 6

Goal
Formulário de novo atendimento (POST
/api/encounters) que nunca cria dois.
```

```
Context
@src/features/admissao/ @INVARIANTES.md

Constraints
- useActionState + trava com useRef
  (UI-04); disabled não é trava
- estados num tipo só, como Resultado
- 400 no campo; 403 e 409 em role=alert
- não altere a API

Done when
- npm run gate verde
- teste: servidor lento, dois cliques,
  um fetch; falha sem o ref
- teste: 403 mostra a mensagem do papel
```

Exercícios

ESSENCIAL No `FormLogin`, troque o estado `enviando` por um `useActionState`. O que acontece com os campos controlados depois de um login com erro?

INTERMEDIÁRIO Faça o botão "Admitir" só habilitar quando os três campos estiverem preenchidos, sem criar estado novo. Que tipo de campo isso exige?

DESAFIO Rode `medicoes/cap06.py` com a rede 3 segundos mais lenta e observe o que a pessoa vê enquanto espera. Proponha o texto e o estado visual que faltam.

Régua: 07, 09; prepara 08. **Invariantes:** UI-04, UI-05. **Tags:** `cap06-antes`, `cap06` (tela) e `cap06` (API). **Prompts:** `prompts/cap06.md`.

Fontes. react.dev: *State as a Snapshot*, *<form>*, *useActionState*, *useRef*. RFC 9110 (409). Saídas em `saidas/c6-*.txt`.



PARTE II · OS FUNDAMENTOS

07

Efeitos: sincronizar com o mundo lá fora

Efeito não é "rodar código depois do render". É manter algo fora do React em dia com o estado, e desfazer isso quando o estado muda.

Terça, 16h. A médica procura "mar" na busca do prontuário. A tela mostra Maria Souza e, um instante depois, João Lima também. João não tem "mar" no nome. Ela recarrega, digita de novo, e o erro não volta.

Bug que aparece e some é quase sempre uma questão de tempo. Aqui, duas respostas do servidor chegaram fora de ordem, e a mais velha venceu. Este capítulo trata do único lugar do React feito para falar com o

mundo lá fora, o efeito, e de como ele erra quando ninguém pensa em limpeza.

7.1

As regras dos Hooks

Funções que começam com `use` são **Hooks**. O React guarda o estado deles pela *ordem* em que são chamados em cada render, por isso eles só podem ser chamados no nível de cima de um componente ou de outro Hook: nunca dentro de `if`, laço ou função comum. A regra `react/rules-of-hooks` do oxlint já vem como erro no template.

7.2

Três tipos de lógica

Todo código de um componente cabe em um de três lugares, e só um deles é um efeito:



Fig. 7.1 — Em cima, a pergunta que decide se você precisa de um efeito. Embaixo, o ciclo de um efeito com limpeza, com a busca deste capítulo como exemplo.

Total e lista filtrada são cálculo. Enviar a admissão é resposta a evento. Buscar o termo atual no servidor é sincronização. A diferença entre os dois últimos é a causa. O envio acontece porque alguém clicou; a busca precisa acontecer porque o termo *é* "mar", não importa como ele chegou a ser. Efeito é para o que precisa ficar em dia com o estado, seja qual for o caminho até ele.

H+ **O pedido ao laboratório**

A enfermaria não faz o exame: pede ao laboratório, que fica fora dela. Se o médico troca o pedido antes do resultado sair, alguém precisa cancelar o pedido antigo, ou o resultado errado chega ao prontuário.

ONDE A ANALOGIA PARA

O laboratório devolve um resultado só. Um efeito pode rodar e se desfazer muitas vezes por minuto, a cada mudança de dependência, e a limpeza roda sempre antes do próximo setup.

7.3

A busca sem limpeza

Na tag `cap07-antes`, a busca do prontuário pede ao servidor, a cada tecla, os pacientes cujo nome contém o termo. A API ganhou o filtro `?q=` na tag `cap07`:

BuscaProntuario.tsx — cap07-antes

```
export function BuscaProntuario() {
  const [termo, setTermo] = useState("");
  const [achados, setAchados] =
    useState<Achado[]>([]);
  const [total, setTotal] = useState(0);

  useEffect(() => {
    if (!termo) return;
    const url = "/api/patients?q="
```

```
      + encodeURIComponent(termo)
      + location.search.replace("?", "&");
    fetch(url)
      .then((r) => r.json())
      .then((dados) => setAchados(dados));
  }, [termo]);

  // "manter o total em dia"
  useEffect(() => {
    setTotal(achados.length);
  }, [achados]);
```

O efeito declara que depende de `termo` e roda a cada mudança. Cada execução dispara um `fetch` e esquece dele. Se a resposta de "ma" chegar depois da de "mar", ela sobrescreve a tela. O `location.search` só existe para o laboratório: repassa à API o parâmetro `?atraso=`, que fora de produção segura a resposta de um termo escolhido.

Este roteiro abre a tela com `?atraso=900:lento:ma`, digita "ma" e, logo depois, "r", e registra cada pedido e cada resposta:

medicoes/cap07.py · Medido

```
== cap07-antes
   30 ms  pede  q=m
   32 ms  pede  q=ma
   35 ms  chega q=m
  196 ms  pede  q=mar
  204 ms  chega q=mar
  945 ms  chega q=ma
  termo na tela: 'mar'
  resultados:    ['Maria Souza', 'João Lima']
```

A resposta de "mar" chegou em 204 ms e mostrou Maria. A de "ma", pedida antes, chegou em 945 ms e trocou a tela pela lista de "ma", com João. O termo na tela continua "mar". Em produção, quem decide a ordem é a rede, não um parâmetro.

O segundo efeito do componente tem outro defeito: copia `achados.length` para um estado `total`. É um valor derivado guardado em dobro, e cada busca custa um render a mais. O oxlint avisa, mas aviso não derruba o portão:

```
terminal · Medido

$ npx oxlint
src/features/prontuario/BuscaProntuario.tsx:25:5
  warning react(set-state-in-effect)
  Calling setState synchronously within an effect
can trigger cascading renders
```

7.4

A correção: limpar e cancelar

Na tag `cap07`, a busca virou um **Hook personalizado**, uma função comum cujo nome começa com `use` e que chama outros Hooks:

```
src/features/prontuario/useBuscaPacientes.ts  cap07

export function useBuscaPacientes(
  termo: string,
) {
  const [achados, setAchados] =
```

```
    useState<Achado[]>([]);

  useEffect(() => {
    if (!termo) return;
    const ctrl = new AbortController();
    const url = "/api/patients?q="
      + encodeURIComponent(termo)
      + location.search.replace("?", "&");
    fetch(url, { signal: ctrl.signal })
      .then((r) => r.json())
      .then((dados) => setAchados(dados))
      .catch((e: Error) => {
        // cancelado de propósito: não é erro
        if (e.name !== "AbortError") throw e;
      });
    return () => ctrl.abort();
  }, [termo]);

  return termo ? achados : [];
}
```

A função devolvida pelo efeito é a **limpeza**. O React a chama antes de rodar o efeito de novo e quando o componente sai da tela. Aqui ela cancela o pedido com `AbortController`: o `fetch` antigo é interrompido e a resposta dele nunca chega ao `setAchados`. O `catch` ignora só o cancelamento, que não é erro.

medicoes/cap07.py · Medido

```
== cap07
   17 ms  pede  q=m
   23 ms  pede  q=ma
   29 ms  cancelado q=m
  188 ms  pede  q=mar
  191 ms  cancelado q=ma
  198 ms  chega q=mar
  termo na tela: 'mar'
  resultados:    ['Maria Souza']
```

Quando o termo virou "mar", a limpeza cancelou "ma" em 191 ms. A documentação oficial mostra outra forma de limpeza, uma variável `ignore` que descarta a resposta velha; ela evita o bug, mas deixa o pedido seguir na rede. O `AbortController` evita o bug e economiza o servidor.

O componente ficou com uma linha para os achados e uma para o total, agora calculado:

src/features/prontuario/BuscaProntuario.tsx cap07

```
…
export function BuscaProntuario() {
  const [termo, setTermo] = useState("");
  const achados = useBuscaPacientes(termo);
  const total = achados.length; // derivado
…
```

REGRAS · UI-06 E UI-07

UI-06: efeito só sincroniza com sistema externo; nunca copia um estado para outro. Verificação: `react/set-state-in-effect` agora como **erro** no `.oxlintrc.json`, junto com `react/exhaustive-deps`.

UI-07: efeito que busca dados cancela o pedido na limpeza. Verificação: o teste da corrida.

7.5

A prova sem servidor

A medição usa a API real. O teste não pode depender da rede, então ele cria uma rede falsa em que nenhum pedido responde sozinho: o teste decide a ordem.

BuscaProntuario.test.tsx — cap07

```
test("resposta velha não sobrescreve a nova",
  async () => {
    const fila = redeControlada({
      ma: ["Maria Souza", "João Lima"],
      mar: ["Maria Souza"],
    });
    render(<BuscaProntuario />);
    const campo = screen.getByLabelText(
      "Buscar no prontuário");
    await userEvent.type(campo, "ma");
    await userEvent.type(campo, "r");
    // "mar" responde primeiro; "ma", depois
    const porTermo = (t: string) => fila.find(
      (p) => p.url.includes(`q=${t}&`)
        || p.url.endsWith(`q=${t}`))!;
    await act(async () =>
```

```
      porTermo("mar").responder());
    await act(async () =>
      porTermo("ma").responder());

    expect(nomes())
      .toEqual(["Maria Souza"]);
    expect(screen.getByText(/1 encontrado/))
      .toBeTruthy();
  });
```

CAP07-ANTES

```
× resposta velha não sobrescreve a nova
→ expected [ 'Maria Souza', 'João Lima' ]
  to deeply equal [ 'Maria Souza' ]
```

CAP07

```
✓ resposta velha não sobrescreve a nova
```

A rede falsa respeita o `signal`: um pedido cancelado rejeita com `AbortError`, como o `fetch` de verdade. Se ela ignorasse o cancelamento, a versão corrigida falharia, e o teste estaria testando a rede falsa, não o componente.

7.6

StrictMode e a limpeza

Em desenvolvimento, o `StrictMode` monta cada componente, desmonta e monta de novo, rodando setup, limpeza e setup. É um teste automático da

limpeza: se o efeito estiver certo, ninguém percebe; se faltar limpeza, aparecem dois pedidos, duas conexões, dois ouvintes de evento. A regra da documentação é que o usuário não deve conseguir distinguir uma montagem de setup, limpeza e setup. Não desligue o `StrictMode` para "resolver" o efeito duplicado: corrija a limpeza.

7.7

useRef, useMemo e useCallback

O `useRef` apareceu no capítulo 6 como trava. Ele guarda um valor que sobrevive aos renders, muda na hora e não provoca render. Serve também para guardar um nó do DOM (`ref={campo}`), por exemplo para dar foco. Não leia nem escreva `ref.current` durante o render: o render precisa ser puro.

`useMemo` guarda o resultado de um cálculo e `useCallback` guarda uma função, ambos até uma dependência mudar. Eles existem por dois motivos: evitar repetir um cálculo caro e manter a mesma referência entre renders, para que um filho memoizado não renderize à toa ou um efeito não rode de novo.

DADO · REACT.DEV, REACT COMPILER 1.0 (OUT. 2025)

O compilador memoiza automaticamente. Para código novo, a documentação recomenda deixar a memoização com ele e usar `useMemo` e `useCallback` como válvula de escape, quando for preciso controle exato. O projeto liga o compilador no capítulo 16, com medição antes e depois.

Por isso o Mini-Prontuário ainda não usa nenhum dos dois; meça antes de usar. Por fim, o `useEffectEvent`, estável desde o React 19.2, cria uma função que o efeito chama e que sempre vê os valores atuais sem entrar nas dependências: por exemplo, registrar a busca em log com o nome de quem está logado sem refazer a busca quando a sessão muda.

7.8

Você talvez não precise de um efeito

A documentação oficial tem uma página inteira sobre efeitos desnecessários. No Mini-Prontuário, eles teriam esta cara:

TENTAÇÃO	NO LUGAR
Efeito que filtra a lista quando a busca muda	Filtre no render (cap. 3)
Efeito que copia `achados.length` para `total`	Calcule no render (este capítulo)
Efeito que envia a admissão quando `enviar` vira `true`	Envie no handler ou na action (cap. 6)
Efeito que zera o formulário quando muda o paciente	Troque a `key` do formulário (cap. 3)
Efeito que avisa o pai que algo mudou	Chame o callback no handler (cap. 5)
Efeito que busca dados	Pode ficar, com limpeza; no cap. 14, uma biblioteca de dados

7.9

Com agentes de IA

ARMADILHA DE IA

É o tema em que código gerado mais erra. Três vícios aparecem juntos: `fetch` em efeito sem limpeza; `setState` no efeito para derivar um valor; e dependência removida da lista "para parar o loop", que esconde o bug em vez de corrigi-lo. Com `set-state-in-effect` e `exhaustive-deps` como erro, os dois últimos não passam do portão; o primeiro, o teste da corrida pega.

PROMPT · AGENTE · CAP 7

```
Goal
Busca do prontuário com "buscando…" e
mensagem quando a rede falhar.

Context
@src/features/prontuario/ @INVARIANTES.md

Constraints
- o hook devolve dados, carregando e
  erro num tipo só
- AbortError não é erro de rede
- não mexa em .oxlintrc.json
- fora de escopo: debounce, cache

Done when
- npm run gate verde
- o teste da corrida passa sem mudar
- teste novo: rede que rejeita mostra
  a mensagem; um cancelamento, não
```

PONTE · DEV DE IA, CAPS. 9 E 13

"Não mexa em `.oxlintrc.json`" existe porque um agente diante de um erro de lint às vezes desliga a regra. Restrições que protegem o portão estão no capítulo 9; a revisão em contexto novo, que pega esse tipo de atalho, no 13.

Exercícios

ESSENCIAL Troque o `AbortController` pela variável `ignore` da documentação e rode o teste da corrida. Passa? Rode `medicoes/cap07.py` e compare as linhas "cancelado".

INTERMEDIÁRIO Coloque o foco no campo de busca quando a tela abre, com `useRef`. Isso precisa de efeito? Por quê?

DESAFIO Acrescente um atraso de 300 ms depois da última tecla antes de buscar (*debounce*), sem biblioteca, como um Hook `useAtrasado(valor, ms)`. Prove com um teste que três teclas rápidas geram um pedido só.

Régua: 10, 11, 12, 13, 36. **Invariantes:** UI-06, UI-07. **Tags:** `cap07-antes`, `cap07` (tela) e `cap07` (API). **Prompts:** `prompts/cap07.md`.

Fontes. react.dev: *Synchronizing with Effects*, *You Might Not Need an Effect*, *useEffectEvent*, *React Compiler*. MDN:

AbortController. Saídas em `saidas/c7-*.txt`.